

SQL Queries, Joins, Aggregation & Index Thinking

This topic is very important for **Java/Spring Boot backend development**, because your application will constantly use SQL to retrieve, filter, join, and summarize database data.

I'll use a **Library Management System** throughout the examples so everything connects together.

1. Library Database Schema

We need:

- Authors
- Books
- Members
- Loans

There are four main entities.

Relationships

Author 1 ————— * Book

Member 1 ————— * Loan

Book 1 ————— * Loan

But there is an important point:

A book can have multiple authors, and an author can write multiple books.

Therefore:

Author ————— Book

requires a join table:

book_authors

Complete relationship:

Authors

|

| 1

|

| *

Books

|

| 1

|

| *

Loans

|

| *

|

| 1

Members

And:

Authors — Books



book_authors

2. Complete Library Database

```
CREATE DATABASE library;
```

```
USE library;
```

```
CREATE TABLE authors (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE books (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    title VARCHAR(200) NOT NULL,  
    isbn VARCHAR(50) NOT NULL UNIQUE,  
    published_year INT,  
    total_copies INT NOT NULL  
);
```

```
CREATE TABLE book_authors (  
    book_id INT NOT NULL,  
    author_id INT NOT NULL,
```

PRIMARY KEY (book_id, author_id),

FOREIGN KEY (book_id)

REFERENCES books(id),

FOREIGN KEY (author_id)

REFERENCES authors(id)

);

CREATE TABLE members (

id INT PRIMARY KEY AUTO_INCREMENT,

name VARCHAR(100) NOT NULL,

email VARCHAR(150) NOT NULL UNIQUE,

join_date DATE NOT NULL

);

CREATE TABLE loans (

id INT PRIMARY KEY AUTO_INCREMENT,

book_id INT NOT NULL,

member_id INT NOT NULL,

loan_date DATE NOT NULL,

due_date DATE NOT NULL,

```
return_date DATE,  
  
FOREIGN KEY (book_id)  
REFERENCES books(id),  
  
FOREIGN KEY (member_id)  
REFERENCES members(id)  
);
```

3. Insert Sample Data

Authors

```
INSERT INTO authors (name)  
VALUES  
('Robert C. Martin'),  
('Joshua Bloch'),  
('James Gosling'),  
('Martin Fowler'),  
('Herbert Schildt');
```

Books

```
INSERT INTO books  
(title, isbn, published_year, total_copies)  
VALUES
```

```
('Clean Code', 'ISBN001', 2008, 5),  
( 'Effective Java', 'ISBN002', 2018, 4),  
( 'Java: The Complete Reference', 'ISBN003', 2020, 6),  
( 'Refactoring', 'ISBN004', 2018, 3),  
( 'Head First Java', 'ISBN005', 2019, 5);
```

Book Authors

```
INSERT INTO book_authors (book_id, author_id)  
VALUES  
(1, 1),  
(2, 2),  
(3, 3),  
(3, 5),  
(4, 4),  
(5, 3);
```

Notice:

Java: The Complete Reference

↓

James Gosling

Herbert Schildt

One book has two authors.

Members

```
INSERT INTO members (name, email, join_date)
```

VALUES

```
('Ali', 'ali@gmail.com', '2026-01-10'),  
( 'Ahmed', 'ahmed@gmail.com', '2026-02-15'),  
( 'Muneeb', 'muneeb@gmail.com', '2026-03-01'),  
( 'Hassan', 'hassan@gmail.com', '2026-04-20'),  
( 'Usman', 'usman@gmail.com', '2026-05-10');
```

Loans

Assume today's date is 2026-08-11.

INSERT INTO loans

(book_id, member_id, loan_date, due_date, return_date)

VALUES

```
(1, 1, '2026-08-01', '2026-08-10', NULL),  
(2, 1, '2026-08-05', '2026-08-15', NULL),  
(3, 2, '2026-08-01', '2026-08-07', NULL),  
(4, 3, '2026-08-03', '2026-08-20', '2026-08-08'),  
(5, 3, '2026-08-05', '2026-08-12', NULL),  
(1, 4, '2026-07-20', '2026-07-30', '2026-07-29');
```

4. SELECT

SELECT retrieves data.

```
SELECT * FROM books;
```

This retrieves every column.

Better in real applications:

```
SELECT id, title, isbn
```

```
FROM books;
```

Best practice

Don't unnecessarily do:

```
SELECT *
```

when you only need:

```
SELECT id, title
```

Why?

Because you're asking the database to return unnecessary data.

5. WHERE

WHERE filters rows.

Books published after 2018

```
SELECT id, title, published_year
```

```
FROM books
```

```
WHERE published_year > 2018;
```

Books with 5 copies

```
SELECT *
```

```
FROM books
```

```
WHERE total_copies = 5;
```

Multiple conditions

```
SELECT *
```



```
FROM books
WHERE published_year >= 2018
AND total_copies >= 5;
```

OR

```
SELECT *
FROM books
WHERE published_year = 2018
OR published_year = 2019;
```

IN

Instead of:

```
WHERE published_year = 2018
OR published_year = 2019
OR published_year = 2020
```

Use:

```
SELECT *
FROM books
WHERE published_year IN (2018, 2019, 2020);
```

6. LIKE

Search text patterns.

```
SELECT *
FROM books
```

WHERE title LIKE '%Java%';

% means any number of characters.

Examples:

'%Java%'

Contains Java.

'Java%'

Starts with Java.

'%Java'

Ends with Java.

7. ORDER BY

Sort results.

Ascending

SELECT *

FROM books

ORDER BY published_year ASC;

Descending

SELECT *

FROM books

ORDER BY published_year DESC;

Usually:

ASC

is default.

8. LIMIT

Restrict the number of rows.

Latest 3 books

```
SELECT *  
FROM books  
ORDER BY published_year DESC  
LIMIT 3;
```

This is extremely common for:

- Top products
- Latest posts
- Latest orders
- Top users
- Pagination

9. COUNT

Counts rows.

```
SELECT COUNT(*) AS total_books  
FROM books;
```

Result:

total_books

5

Number of members

```
SELECT COUNT(*) AS total_members  
FROM members;
```

10. SUM

Adds values.

Suppose we want total copies:

```
SELECT SUM(total_copies) AS total_copies  
FROM books;
```

11. AVG

Average value.

```
SELECT AVG(total_copies) AS average_copies  
FROM books;
```

12. MIN

Smallest value.

```
SELECT MIN(published_year) AS oldest_book  
FROM books;
```

13. MAX

Largest value.

```
SELECT MAX(published_year) AS newest_book
```

FROM books;

14. GROUP BY

GROUP BY creates groups so aggregate functions can calculate something for each group.

For example:

How many books does each author have?

SELECT

author_id,

COUNT(*) AS total_books

FROM book_authors

GROUP BY author_id;

Result conceptually:

author_id | total_books

-----|-----

1 | 1

2 | 1

3 | 2

4 | 1

5 | 1

15. GROUP BY With JOIN

We don't want IDs. We want author names.

```

SELECT
    a.name,
    COUNT(ba.book\_id) AS total_books
FROM authors a
JOIN book_authors ba
    ON a.id = ba.author\_id
GROUP BY a.id, a.name;

```

Result:

name	total_books
----- -----	
Robert C. Martin	1
Joshua Bloch	1
James Gosling	2
Martin Fowler	1
Herbert Schildt	1

16. HAVING

This is extremely important.

HAVING filters **groups**.

For example:

Find authors who have written more than one book.

```

SELECT

```

```
a.name,  
COUNT(ba.book_id) AS total_books  
FROM authors a  
JOIN book_authors ba  
ON a.id = ba.author_id  
GROUP BY a.id, a.name  
HAVING COUNT(ba.book_id) > 1;
```

Result:

James Gosling | 2

17. WHERE vs HAVING

This is a **very common** interview question.

WHERE

Filters individual rows **before grouping**.

```
SELECT *  
FROM books  
WHERE published_year >= 2018;
```

HAVING

Filters groups **after GROUP BY**.

```
SELECT  
author_id,  
COUNT(*) AS total_books
```

```
FROM book_authors  
GROUP BY author_id  
HAVING COUNT(*) > 1;
```

Easy memory trick

WHERE

↓

Filter rows

GROUP BY

↓

Create groups

HAVING

↓

Filter groups

Interview answer

WHERE filters individual rows before grouping, while HAVING filters the groups produced by GROUP BY. Aggregate functions are commonly used in HAVING.

18. INNER JOIN

INNER JOIN returns only matching rows from both tables.

Example:


```

SELECT
    l.id,
    m.name AS member,
    b.title AS book
FROM loans l
INNER JOIN members m
    ON l.member_id = m.id
INNER JOIN books b
    ON l.book_id = b.id;

```

Result:

loan | member | book

-----|-----|-----

1	Ali	Clean Code
2	Ali	Effective Java
3	Ahmed	Java: The Complete Reference
4	Muneeb	Refactoring
5	Muneeb	Head First Java
6	Hassan	Clean Code

All loans have valid members and books, so all rows match.

19. LEFT JOIN

LEFT JOIN returns:

All rows from the left table + matching rows from the right table.

Suppose we want **all members**, including members who have never borrowed a book.

SELECT

[m.name](#),

[l.id](#) AS loan_id

FROM members m

LEFT JOIN loans l

ON [m.id](#) = l.member_id;

Our data has:

Ali → loans

Ahmed → loan

Muneeb → loans

Hassan → loan

Usman → NULL

Usman is still returned because we started with:

FROM members m

and used:

LEFT JOIN

20. INNER JOIN vs LEFT JOIN

INNER JOIN	LEFT JOIN
------------	-----------

Only matching rows	All left-table rows
Non-matching left rows disappear	Non-matching left rows remain
Useful when relationship is required	Useful for optional relationships

Example:

Members

Ali

Ahmed

Muneeb

Hassan

Usman

Loans:

Ali

Ahmed

Muneeb

Hassan

INNER JOIN

Ali

Ahmed

Muneeb

Hassan

LEFT JOIN

Ali

Ahmed

Muneeb

Hassan

Usman

21. LEFT JOIN With NULL

Find members who have never borrowed a book:

```
SELECT
```

```
    m.id,
```

```
    m.name
```

```
FROM members m
```

```
LEFT JOIN loans l
```

```
    ON m.id = l.member_id
```

```
WHERE l.id IS NULL;
```

Result:

Usman

This is a very useful SQL pattern.

22. Active Loans

An active loan is one that hasn't been returned.

We designed:

return_date = NULL

for active loans.

Query:

```
SELECT *
```

```
FROM loans
```

```
WHERE return_date IS NULL;
```

Better, with book and member information:

```
SELECT
```

```
    l.id,
```

```
    m.name AS member,
```

```
    b.title,
```

```
    l.loan\_date,
```

```
    l.due_date
```

```
FROM loans l
```

```
JOIN members m
```

```
    ON l.member_id = m.id
```

```
JOIN books b
```

```
    ON l.book\_id = b.id
```

```
WHERE l.return_date IS NULL;
```

23. Overdue Books

An overdue book:

return_date IS NULL

AND due_date < CURRENT_DATE

Query:

SELECT

[l.id](#),

[m.name](#) AS member,

b.title,

l.due_date

FROM loans l

JOIN members m

ON l.member_id = [m.id](#)

JOIN books b

ON [l.book_id](#) = [b.id](#)

WHERE l.return_date IS NULL

AND l.due_date < CURRENT_DATE;

As of 2026-08-11, this would identify loans due before August 11 that haven't been returned.

24. Member Borrowing Count

Question:

How many books has each member borrowed?

SELECT

[m.id](#),

```
m.name,  
COUNT(l.id) AS borrowing_count  
FROM members m  
LEFT JOIN loans l  
ON m.id = l.member_id  
GROUP BY m.id, m.name;
```

Why LEFT JOIN?

Because we want members with **zero loans** too.

If we used INNER JOIN, members who never borrowed anything would disappear.

25. Members With More Than One Loan

```
SELECT  
m.id,  
m.name,  
COUNT(l.id) AS borrowing_count  
FROM members m  
JOIN loans l  
ON m.id = l.member_id  
GROUP BY m.id, m.name  
HAVING COUNT(l.id) > 1;
```

Notice:

WHERE

isn't appropriate for:

COUNT([l.id](#)) > 1

because COUNT() is an aggregate.

We use:

HAVING

26. Top Authors

Question:

Find the authors with the most books.

SELECT

[a.id](#),

[a.name](#),

COUNT([ba.book_id](#)) AS total_books

FROM authors a

JOIN book_authors ba

ON [a.id](#) = [ba.author_id](#)

GROUP BY [a.id](#), [a.name](#)

ORDER BY total_books DESC

LIMIT 3;

This combines:

JOIN

GROUP BY

COUNT

ORDER BY

LIMIT

Very common interview-style query.

27. Subqueries

A **subquery** is a query inside another query.

Example:

Find books published after the average publication year.

First:

```
SELECT AVG(published_year)
```

```
FROM books;
```

Then:

```
SELECT *
```

```
FROM books
```

```
WHERE published_year >
```

```
(
```

```
    SELECT AVG(published_year)
```

```
    FROM books
```

```
);
```

The inner query runs to produce a value.

Conceptually:

Outer query



published_year > ?



Inner query

AVG(year)

28. Another Subquery Example

Question:

Find members who have borrowed at least one book.

```
SELECT *
```

```
FROM members
```

```
WHERE id IN (
```

```
    SELECT member_id
```

```
    FROM loans
```

```
);
```

The inner query returns member IDs:

1

2

3

4

The outer query finds those members.

29. Subquery vs JOIN

Both can sometimes solve the same problem.

Subquery:

```
SELECT *  
FROM members  
WHERE id IN (  
    SELECT member_id  
    FROM loans  
);
```

JOIN:

```
SELECT DISTINCT m.*  
FROM members m  
JOIN loans l  
    ON m.id = l.member_id;
```

Neither is universally “better.” The appropriate choice depends on the problem, database optimizer, indexes, and query structure.

30. Important JOIN Mistake: Row Multiplication

This is a very important practical concept.

Suppose:

Author A

↓

2 books

and each book has:

3 loans

If you join authors → books → loans, you can get:

2 books × 3 loans = 6 rows

The author appears six times.

This can produce incorrect counts if you're not careful.

For example, a naive query like:

```
SELECT
```

```
    a.name,
```

```
    COUNT(*)
```

```
FROM authors a
```

```
JOIN book_authors ba
```

```
    ON a.id = ba.author\_id
```

```
JOIN loans l
```

```
    ON l.book\_id = ba.book\_id
```

```
GROUP BY a.id, a.name;
```

is counting **joined rows**, not necessarily unique books.

If you want unique books:

```
COUNT(DISTINCT ba.book\_id)
```

If you want unique loans:

```
COUNT(DISTINCT l.id)
```

Important idea

Always understand the number of rows each JOIN can produce before applying COUNT, SUM, etc.

31. Indexes

An **index** helps the database find rows faster without scanning the entire table.

Imagine:

books

5 rows

Scanning five rows isn't a big deal.

But imagine:

books

10,000,000 rows

and you run:

```
SELECT *
```

```
FROM books
```

```
WHERE isbn = 'ISBN001';
```

Without an appropriate index, the database may need to inspect many rows.

With an index on isbn, it can locate matching values much more efficiently.

32. Creating an Index

```
CREATE INDEX idx_books_isbn
```

```
ON books(isbn);
```

Now:

```
SELECT *
```

```
FROM books
```

```
WHERE isbn = 'ISBN001';
```

can benefit from that index.

33. Index on Foreign Key Columns

Foreign keys are frequently used in:

```
JOIN
```

```
WHERE
```

For example:

```
SELECT *
```

```
FROM loans
```

```
WHERE member_id = 1;
```

An index can help:

```
CREATE INDEX idx_loans_member_id
```

```
ON loans(member_id);
```

Similarly:

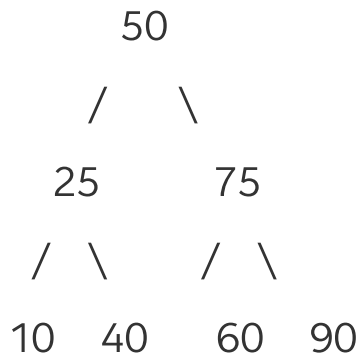
```
CREATE INDEX idx_loans_book_id
```

```
ON loans(book_id);
```

34. What Is a B-tree?

At beginner level, think of a B-tree index as a **balanced tree structure that helps the database quickly navigate to values instead of checking every row one by one.**

Conceptually:



The database can navigate through the tree rather than scanning every value.

B-tree-style indexes are useful for queries involving:

=

>

<

>=

<=

BETWEEN

ORDER BY

depending on the index and query structure.

You don't need to implement a B-tree yourself as a beginner backend developer. You mainly need to understand **why indexes exist and when they help.**

35. Index Trade-Off

Indexes aren't free.

They can:

Improve

SELECT

WHERE

JOIN

ORDER BY

But increase cost of

INSERT

UPDATE

DELETE

because the database may need to maintain the index.

Indexes also consume storage.

Therefore:

Don't create an index on every column.

36. Which Columns Might Need Indexes?

Consider:

members

id

name

email

loans

id

book_id

member_id

loan_date

due_date

return_date

Good candidates may include:

[members.email](#)

loans.member_id

[loans.book_id](#)

loans.due_date

depending on actual query patterns.

email is particularly common for login/search:

SELECT id, name

FROM members

WHERE email = '[muneeb@gmail.com](#)';

member_id is useful for:

SELECT *

FROM loans

WHERE member_id = 3;

37. Composite Index

Sometimes queries filter by multiple columns.

Example:

```
SELECT *  
FROM loans  
WHERE member_id = 3  
AND return_date IS NULL;
```

You might consider:

```
CREATE INDEX idx_loans_member_return  
ON loans(member_id, return_date);
```

This is a **composite index**.

The order of columns matters when designing composite indexes.

38. Don't Index Everything

Bad approach:

Index name

Index email

Index join_date

Index every column

Index everything

Instead ask:

Which columns are frequently used for filtering, joining, sorting, or uniqueness?

For example:

WHERE member_id = ?

frequently?

Consider an index.

WHERE email = ?

frequently?

Consider an index.

39. Application Query Best Practice

Avoid:

SELECT *

FROM books;

if your application only needs:

id

title

Use:

SELECT id, title

FROM books;

Why?

- Less data transferred
- Less memory usage
- Clearer intent
- Better application efficiency
- Avoids unnecessarily retrieving sensitive/unneeded columns

40. Five Assessment Queries

Here are five interview-style exercises based on our library schema.

Question 1

Find all books published after 2018.

Answer

```
SELECT id, title, published_year
FROM books
WHERE published_year > 2018;
```

Question 2

Find all currently active loans with member and book names.

Answer

```
SELECT
    l.id,
    m.name AS member,
    b.title AS book,
    l.due_date
FROM loans l
JOIN members m
    ON l.member_id = m.id
JOIN books b
```

```
ON l.book\_id = b.id  
WHERE l.return_date IS NULL;
```

Question 3

Find members who have borrowed more than one book.

Answer

```
SELECT  
    m.id,  
    m.name,  
    COUNT(l.id) AS total_loans  
FROM members m  
JOIN loans l  
    ON m.id = l.member_id  
GROUP BY m.id, m.name  
HAVING COUNT(l.id) > 1;
```

Question 4

Find the three authors with the most books.

Answer

```
SELECT  
    a.id,  
    a.name,
```

```
COUNT(ba.book\_id) AS total_books
FROM authors a
JOIN book_authors ba
ON a.id = ba.author\_id
GROUP BY a.id, a.name
ORDER BY total_books DESC
LIMIT 3;
```

Question 5

Find members who have never borrowed a book.

Answer

```
SELECT
  m.id,
  m.name
FROM members m
LEFT JOIN loans l
ON m.id = l.member_id
WHERE l.id IS NULL;
```

41. Interview Questions

Q1. What is

```
SELECT
```

?

Retrieves data from one or more tables.

Q2. What does

WHERE

do?

Filters rows before grouping and aggregation.

Q3. What does

ORDER BY

do?

Sorts query results.

ORDER BY name ASC;

or:

ORDER BY name DESC;

Q4. What does

LIMIT

do?

Restricts the number of rows returned.

Q5. What are aggregate functions?

Functions that calculate a result from multiple rows.

COUNT

SUM

AVG

MIN

MAX

Q6. What is GROUP BY?

GROUP BY groups rows based on one or more columns so aggregate calculations can be performed for each group.

Q7. WHERE vs HAVING?

Interview answer:

WHERE filters individual rows before grouping, while HAVING filters groups after GROUP BY. HAVING is commonly used with aggregate functions such as COUNT() or SUM().

Q8. INNER JOIN vs LEFT JOIN?

INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table; unmatched right-side values become NULL.

Q9. When would you use LEFT JOIN?

For optional relationships.

Example:

Find all members, including members who haven't borrowed any books.

Q10. What is a subquery?

A query nested inside another SQL query.

Example:

```
SELECT *  
FROM books  
WHERE published_year > (  
    SELECT AVG(published_year)  
    FROM books  
);
```

Q11. What is an index?

An index is a database data structure that helps the database locate and access rows more efficiently for supported queries.

Q12. Why not create indexes on every column?

Because indexes consume storage and can slow down INSERT, UPDATE, and DELETE operations due to index maintenance.

Q13. What is a B-tree index?

A B-tree index is a balanced tree-based structure that allows the database to efficiently navigate to indexed values instead of scanning every row.

Q14. What is row multiplication in JOINS?

When joining tables, one row from one table can match multiple rows from another table, causing multiple result rows. This can produce incorrect aggregate results if you don't account for it.

42. The SQL Execution Order You Should Know

This is another **excellent interview point**.

Although we write:

SELECT

FROM

WHERE

GROUP BY

HAVING

ORDER BY

LIMIT

the logical processing order is approximately:

FROM

↓

JOIN

↓

WHERE

↓

GROUP BY

↓

HAVING

↓

SELECT

↓

ORDER BY

↓

LIMIT

Remember:

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT →
ORDER BY → LIMIT

This makes WHERE vs HAVING much easier to understand.

43. Final Cheat Sheet

Concept	Purpose
SELECT	Retrieve data
WHERE	Filter rows
ORDER BY	Sort results
LIMIT	Limit rows
COUNT()	Count
SUM()	Total
AVG()	Average
MIN()	Minimum

MAX()	Maximum
GROUP BY	Create groups
HAVING	Filter groups
INNER JOIN	Matching records
LEFT JOIN	All left + matching right
Subquery	Query inside query
Index	Faster lookup/access for supported queries
B-tree	Tree-based index structure
DISTINCT	Remove duplicate result values
IS NULL	Check for NULL

Most important things for your interview

If you remember only these, remember:

WHERE → filters rows

GROUP BY → creates groups

HAVING → filters groups

INNER JOIN → only matching records

LEFT JOIN → all left records + matches

COUNT/SUM/AVG/MIN/MAX → aggregation

Many-to-many → join table

Index → faster reads, but costs storage/write performance

JOIN → understand possible row multiplication

SELECT only what the application actually needs

And the most important mental model:

FROM / JOIN



Get the data



WHERE



Filter rows



GROUP BY



Make groups



HAVING



Filter groups



SELECT



Choose output



ORDER BY



Sort



LIMIT



Restrict result